Appendix A. Key parameters for the widely-used NumPy functions:

1. **np.ones(shape, dtype=None, order='C'):**
    - shape: Shape of the array.
    - dtype: Data type of the array elements.
    - order: Whether to store the array in row-major (C-style) or column-major (Fortran-style) order.
2. **np.zeros(shape, dtype=float, order='C'):**
    - shape: Shape of the array.
    - dtype: Data type of the array elements.
    - order: Whether to store the array in row-major (C-style) or column-major (Fortran-style) order.
3. **np.arange([start,]stop, [step,]dtype=None):**
    - start: Start of the interval (inclusive).
    - stop: End of the interval (exclusive).
    - step: Spacing between values.
    - dtype: Data type of the array elements.
4. **np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None):**
    - start: Start of the sequence.
    - stop: End of the sequence.
    - num: Number of samples to generate.
    - endpoint: Whether to include stop in the sequence.
    - retstep: Whether to return the step between samples.
    - dtype: Data type of the array elements.
5. **np.repeat(a, repeats, axis=None):**
    - a: Input array.
    - repeats: The number of repetitions for each element.
    - axis: The axis along which to repeat values.
6. **np.tile(A, reps):**
    - A: Input array.
    - reps: The number of repetitions of A along each axis.
7. **np.random.random(size=None):**
    - size: Output shape.
8. **np.random.randn(d0, d1, ..., dn):**
    - d0, d1, ..., dn: Dimensions of the returned array.
9. **np.random.randint(low, high=None, size=None, dtype=int):**
    - low: Lowest (signed) integer to be drawn from the distribution.
    - high: If provided, one above the largest (signed) integer to be drawn.
    - size: Output shape.
    - dtype: Data type of the output.
10. **np.random.normal(loc=0.0, scale=1.0, size=None):**
    - loc: Mean of the distribution.
    - scale: Standard deviation of the distribution.
    - size: Output shape.
11. **np.sort(a, axis=-1, kind=None, order=None):**
    - a: Array to be sorted.
    - axis: Axis along which to sort.

- kind: Sorting algorithm.
- order: When a is a structured array, this specifies which fields to compare first.

12. np.argsort(a, axis=-1, kind=None, order=None):

- a: Array to be sorted.
- axis: Axis along which to sort.
- kind: Sorting algorithm.
- order: When a is a structured array, this specifies which fields to compare first.

13. np.hstack(tup):

- tup: Sequence of arrays to stack horizontally.

14. **np.vstack(tup):**

- tup: Sequence of arrays to stack vertically.

15. **np.concatenate(arrays, axis=0, out=None):**

- arrays: Sequence of arrays to concatenate.
- axis: Axis along which arrays will be joined.
- out: Output array.

16. **np.append(arr, values, axis=None):**

- arr: Input array.
- values: Values to append to arr.
- axis: Axis along which to append.

17. np.reshape(a, newshape, order='C'):

- a: Array to be reshaped.
- newshape: New shape.
- order: Whether to read/write the elements using C-style or Fortran-style index order.

18. np.mean(a, axis=None, dtype=None, out=None, keepdims=<no value>):

- a: Array containing numbers whose mean is desired.
- axis: Axis or axes along which the means are computed.
- dtype: Data type to use in computing the mean.
- out: Output array.
- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.

19. np.average(a, axis=None, weights=None, returned=False):

- a: Array containing numbers.
- axis: Axis or axes along which to average a.
- weights: An array of weights associated with the values in a.
- returned: If True, return the tuple (average, sum_of_weights).

20. np.std(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>):

- a: Array containing numbers.
- axis: Axis or axes along which the standard deviation is computed.
- dtype: Data type to use in computing the standard deviation.
- out: Output array.
- ddof: Delta degrees of freedom. The divisor used in calculations is N - ddof, where N represents the number of elements.

- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.

21. np.min(a, axis=None, out=None, keepdims=<no value>):

- a: Array containing numbers.
- axis: Axis or axes along which to find the minimum value.
- out: Output array.
- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.

22. np.max(a, axis=None, out=None, keepdims=<no value>):

- a: Array containing numbers.
- axis: Axis or axes along which to find the maximum value.
- out: Output array.
- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.

23. np.argmin(a, axis=None, out=None):

- a: Array in which to find the minimum value.
- axis: Axis along which to operate.
- out: Output array.

24. np.argmax(a, axis=None, out=None):

- a: Array in which to find the maximum value.
- axis: Axis along which to operate.
- out: Output array.

25. np.ptp(a, axis=None, out=None):

- a: Array containing numbers.
- axis: Axis or axes along which to find the peak-to-peak (maximum - minimum) value.
- out: Output array.

26. np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>):

- a: Array containing numbers to be summed.
- axis: Axis or axes along which the sum is computed.
- dtype: Data type to use in computing the sum.
- out: Output array.
- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.
- initial: Value at which to start the sum.

27. np.median(a, axis=None, out=None, overwrite_input=False, keepdims=False):

- a: Array containing numbers.
- axis: Axis or axes along which to compute the median.
- out: Output array.
- overwrite_input: If True, the input array is used as working storage for the calculation.
- keepdims: If this is set to True, the axes which are reduced are left in the result as dimensions with size one.

Appendix B. Key parameters for the widely-used Pandas functions:

1. **pd.read_excel(io, sheet_name=None, header=0, usecols=None, skiprows=0, nrows=None, dtype=None)**
    - io: The file path or buffer where the Excel file is located.
    - sheet_name: Specifies the sheet name or number to read from the Excel file.
    - header: Specifies which row to use as column names.
    - usecols: A list of columns to read from the Excel file.
    - skiprows: Number of rows to skip before reading the data.
    - nrows: Number of rows to read.
    - dtype: Data type to force for certain columns.
2. **pd.read_csv(filepath_or_buffer, sep=',', header='infer', usecols=None, skiprows=0, nrows=None, dtype=None)**
    - filepath_or_buffer: The file path or buffer where the CSV file is located.
    - sep: Delimiter to use for separating columns.
    - header: Specifies which row to use as column names.
    - usecols: A list of columns to read from the CSV file.
    - skiprows: Number of rows to skip before reading the data.
    - nrows: Number of rows to read.
    - dtype: Data type to force for certain columns.
3. **pd.melt(data, id_vars=None, value_vars=None, var_name='variable', value_name='value')**
    - data: The DataFrame to melt.
    - id_vars: Columns to use as identifier variables (columns to remain in the reshaped DataFrame).
    - value_vars: Columns to unpivot (melt) from wide to long format.
    - var_name: Name to use for the variable column.
    - value_name: Name to use for the value column.
4. **pd.pivot_table(data, index=None, columns=None, values=None, aggfunc='mean', fill_value=None)**
    - data: The DataFrame to pivot.
    - index: Columns to group by (can be a single column name or a list of column names).
    - columns: Column to pivot (can be a single column name).
    - values: Column(s) to aggregate (can be a single column name or a list of column names).
    - aggfunc: Function(s) to use for aggregating values.
5. **pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, suffixes=('_x', '_y'))**
    - left: DataFrame object to merge on the left side.
    - right: DataFrame object to merge on the right side.
    - how: Type of merge to be performed (e.g., 'inner', 'outer', 'left', 'right').
    - on: Column(s) to join on (can be a single column name or a list of column names).
    - left_on and right_on: Columns in the left and right DataFrame to join on if they have different names.
    - suffixes: Suffixes to apply to overlapping column names after the merge.

6. **pd.DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, observed=False, dropna=True)**
    - by: Column(s) to group by (string, list, function, or array-like).
    - axis: Axis to group (0 for rows, 1 for columns). Default: 0.
    - level: For hierarchical indexes, specify the level(s) to group.
    - as_index: If True, group labels become the index of the aggregated result. Default: True.
    - sort: Sort group keys. Default: True.
    - group_keys: Add group keys to index when calling apply. Default: True.
    - observed: For categorical data, use only observed categories. Default: False.
    - dropna: If True, exclude NA/null groups. Default: True.
7. **GroupBy.agg(func=None, axis=0, *args, **kwargs)**
    - func: Aggregation function(s) (e.g., 'sum', 'mean', np.std, or a list/dict of functions).
    - axis: Axis to apply the function (0 for rows, 1 for columns). Default: 0.
    - *args/**kwargs: Additional arguments for the aggregation function.
8. **pd.DataFrame.reset_index(level=None, drop=False, inplace=False, col_level=0, col_fill='')**
    - level: Specific index level(s) to reset (for hierarchical indexes).
    - drop: If True, discard the index instead of adding it as a column. Default: False.
    - inplace: Modify the DataFrame in place. Default: False.
    - col_level: If columns are hierarchical, insert the index into this level.
    - col_fill: Label to use for the reset index column if col_level is specified.